

Divide et impera

Prima di discutere l'algoritmo ci tocca parlare della tecnica che quest'ultimo sfrutta: **divide et impera**. Caratterizzata da tre passaggi:

- **divide**: il problema iniziale **P** di dimensione n viene suddiviso in k sotto-problemi di dimensione più piccola di n , indicato come $\frac{n}{b}$.
- risoluzione **ricorsiva** dei sotto problemi; se la ricorsione arriva ad un caso base allora risolviamo direttamente.
- **impera**: combiniamo le soluzioni di ciascun sotto-problema per ottenere la soluzione del problema originale **P**.

Il costo computazionale di un qualsiasi algoritmo *divide et impera* prende questa forma:

$$T(n) = k \cdot T\left(\frac{n}{b}\right) + f(n)$$

Dove k è il numero di sotto-problemi generati, $\frac{n}{b}$ la loro dimensione e $f(n)$ il costo per dividere e combinare i problemi; questa formula prende la stessa struttura di quella studiata nel capitolo sugli *algoritmi ricorsivi*, poichè il *divide et impera* è un applicazione diretta del paradigma di ricorsione.

Quicksort non in-locò

Algorithm: Ordina l'array

```
1 function QUICKSORT(array A)
2   x ← elemento di A;
3   array A1 ← {elementi di A minori di x} ;
4   array A2 ← {elementi di A maggiori di x} ;
5   array A3 ← {valori di A uguali a x} ;
6   if |A1| > 1 then
7     QUICKSORT(A1)
8   if |A2| > 1 then
9     QUICKSORT(A2)
10  copiamo A1 e A2 in A
```

L'algoritmo **QuickSort** è un algoritmo di **ordinamento** che sfrutta la metodologia *divide et impera*:

- **divide**: l'array in tanti sotto-array in base al valore del *perno* (ovvero la variabile x)
- risolve ricorsivamente A_1 e A_2

- **impera**: concatena le due sottosequenze ordinate

Il passaggio del **divide** è più complesso di quello **impera**, al contrario del **MergeSort**.

L'algoritmo però non è molto efficiente per un grande proble di costo in **memoria**, infatti ad ogni chiamata vengono allocati nuovi sottoarray A_1, A_2 e A_3 .

Per questo motivo è nato il **QuickSort in-loco**.

Quicksort in-loco

Algorithm: Ordina l'array, versione in-loco

```

1 function QUICKSORT(array A, int i, int f)
2   if  $i \geq f$  then
3     return;
4    $m \leftarrow \text{PARTITION}(A, i, f)$  ;
5   QUICKSORT(A, i,  $m - 1$ ) ;
6   QUICKSORT(A,  $m + 1$ , f) ;
7 function PARTITION(array A, int i, int f)
8    $x \leftarrow A[i]$ ;
9    $\text{inf} \leftarrow i$ ;
10   $\text{sup} \leftarrow f + 1$ ;

```
